

AA 2025-2026 - Metodi del Calcolo Scientifico - Progetto 1

Benbouzid Hicham 894680, Shi Le Yang 894536, Zen Elena 895355

Contents

1	Introduzione	2
2	Metodologia di Calcolo e Dataset Analizzato	3
2.1	Il Dataset: Matrici Sparse Simmetriche e Definite Positive	3
2.2	Impostazione del Sistema Lineare	4
2.3	Metriche di Valutazione	4
3	Il Motore Computazionale: Architettura Condivisa tra MATLAB e Python	5
3.1	Il Core Algoritmico: CHOLMOD	5
3.2	Integrazione nell'ambiente MATLAB	5
3.3	Implementazione in ambiente Python: scikit-sparse	6
3.3.1	Gestione dei Dati in Python	6
3.4	Implicazioni per il Benchmarking	7
3.5	Considerazioni su Usabilità e Documentazione	7
4	Risultati Sperimentali e Analisi dei Benchmark	9
4.1	Performance su Windows	10
4.1.1	MATLAB	10
4.1.2	Python	11
4.2	Performance su Linux	13
4.2.1	MATLAB	13
4.2.2	Python	14
4.3	Analisi Comparativa	15
4.3.1	MATLAB: Windows vs Linux	15
4.3.2	Python: Windows vs Linux	17
4.3.3	Windows: MATLAB vs Python	19
4.3.4	Linux: MATLAB vs Python	21
5	Conclusioni	25
5.1	Quadro di sintesi degli andamenti incrociati	25
5.2	Vincoli architetturali e impatto sulle risorse	25
5.3	Decisione operativa	26
6	Listato del codice	27
6.1	MATLAB	27
6.1.1	Script principale	27
6.1.2	Profiler	31
6.2	Python	32

Chapter 1

Introduzione

Il presente documento illustra i risultati di un'analisi comparativa incentrata sull'implementazione del metodo di **Cholesky** per la risoluzione di sistemi lineari caratterizzati da matrici **sparse**, **simmetriche** e **definite positive**. L'obiettivo principale di questo studio è valutare e confrontare l'efficienza di ambienti di programmazione **open source** rispetto all'implementazione proprietaria fornita da **MATLAB**.

L'analisi risponde alla necessità strategica aziendale di dotarsi di un ambiente di programmazione capace di gestire e risolvere in modo efficiente sistemi lineari di grandi dimensioni. L'alternativa decisionale analizzata si pone tra il mantenimento di un software proprietario e a pagamento come MATLAB, e l'adozione di una libreria open source alternativa. Tra le diverse alternative disponibili nel panorama open source, è stato deciso di adottare **Python** come linguaggio di riferimento per il confronto.

Per fornire una valutazione completa e oggettiva, il confronto tecnico è stato condotto operando sulla **medesima macchina fisica**, ma testando le prestazioni su due diversi sistemi operativi: **Windows** e **Linux**. I parametri chiave presi in esame per misurare e confrontare le performance dei solutori includono:

- **Velocità di esecuzione:** il tempo di calcolo effettivo necessario per ottenere la soluzione del sistema.
- **Accuratezza:** valutata calcolando l'errore relativo tra la soluzione calcolata e una soluzione esatta nota a priori.
- **Occupazione di memoria:** l'impiego di risorse richiesto dal sistema durante l'intera fase di risoluzione delle matrici.
- **Usabilità:** la facilità d'uso generale degli strumenti e la qualità della documentazione a supporto degli sviluppatori.

In sintesi, questa relazione tecnica mira a fornire un quadro chiaro basato sui dati per supportare una decisione operativa: stabilire se sia più conveniente e sicuro affidarsi all'ecosistema di MATLAB, giustificandone i costi di licenza, oppure se il passaggio a una soluzione open source come Python rappresenti un'alternativa valida e performante. Contestualmente, lo studio permetterà di individuare quale tra gli ambienti Linux e Windows risulti ottimale per questo specifico carico di lavoro.

Chapter 2

Metodologia di Calcolo e Dataset Analizzato

Per condurre un confronto rigoroso tra l'ambiente proprietario MATLAB e la soluzione open source selezionata, è stata definita una metodologia di *benchmarking* standardizzata. I test si concentrano sulla risoluzione di sistemi lineari strutturati nella forma $Ax = b$, impiegando dataset rappresentativi di scenari operativi reali e metriche di valutazione oggettive.

2.1 Il Dataset: Matrici Sparse Simmetriche e Definite Positive

L'analisi si focalizza esclusivamente sull'elaborazione di matrici sparse, simmetriche e definite positive. Per garantire che il test rifletta carichi di lavoro concreti, i dati di input sono stati estratti dalla **SuiteSparse Matrix Collection**. Questo database rappresenta uno standard industriale, raccogliendo matrici sparse derivanti da applicazioni pratiche in campi quali l'ingegneria strutturale, la fluidodinamica, l'elettromagnetismo, la termodinamica, la computer vision e l'analisi dei network.

Nello specifico, per stressare i solutori su diverse scale di complessità, sono state selezionate e analizzate le seguenti matrici in ordine di dimensione crescente:

- Flan_1565
- StocF-1465
- cfd2
- cfd1
- G3_circuit
- parabolic_fem
- apache2
- shallow_water1
- ex15

Nota operativa sui limiti Hardware: Durante la fase preliminare di esecuzione, sono emerse criticità infrastrutturali legate alla macchina utilizzata per il benchmarking. Nello specifico, la matrice **Flan_1565** si è rivelata **troppo grande**: l'occupazione ha ecceduto la capacità della memoria RAM fisica disponibile, rendendo impossibile il completamento del calcolo. Inoltre, l'elaborazione della matrice **StocF-1465** in ambiente **Windows** ha innescato un pesante fenomeno di **swapping** della memoria su VRAM. Poiché questo comportamento anomalo inficia fortemente l'affidabilità delle misurazioni prestazionali (in particolare quelle temporali), si è deciso di **escludere** definitivamente entrambe queste matrici dal dataset analitico finale.

2.2 Impostazione del Sistema Lineare

Per poter misurare con precisione assoluta l'accuratezza degli algoritmi, i sistemi lineari sono stati costruiti artificialmente partendo da una soluzione **esatta** nota a priori. Per ogni matrice A del dataset, è stato definito un vettore soluzione esatto x_e le cui componenti sono tutte pari a 1 ($x_e = [1, 1, \dots, 1]^T$). Conseguentemente, il termine noto b è stato calcolato tramite la moltiplicazione matrice-vettore $b = A \cdot x_e$. Il solutore in fase di test riceve in input esclusivamente A e b , ed elabora la soluzione calcolata x , la quale viene poi confrontata con la soluzione esatta x_e per valutarne le **performance algoritmiche**.

2.3 Metriche di Valutazione

L'efficienza operativa e la stabilità numerica dei solutori basati sul metodo di Cholesky sono state valutate misurando tre parametri fondamentali per ogni singola matrice elaborata:

1. Velocità di Esecuzione (**Tempo**): Misurazione del tempo di calcolo netto necessario all'algoritmo per elaborare e restituire la soluzione x .
2. Accuratezza (**Errore Relativo**): Calcolo della discrepanza tra la soluzione elaborata x e la soluzione esatta x_e . La misurazione avviene tramite il calcolo dell'errore relativo basato sulla norma euclidea (o norma 2) dei vettori, secondo la formula:

$$Errore\ relativo = \frac{\|x - x_e\|_2}{\|x_e\|_2}$$

dove $\|v\|_2$ rappresenta la norma euclidea del vettore v .

3. Occupazione di Memoria (**RAM**): La risorsa di memoria necessaria per risolvere il sistema. Per questa metrica si è scelto di registrare il **picco massimo** di memoria RAM utilizzato dal processo durante l'intera fase di elaborazione e risoluzione, garantendo così una valutazione accurata del reale impatto sui requisiti hardware della macchina.

Chapter 3

Il Motore Computazionale: Architettura Condivisa tra MATLAB e Python

Per garantire una corretta interpretazione dei risultati di benchmarking, è fondamentale analizzare l'infrastruttura algoritmica sottostante i due ambienti operativi. Sebbene MATLAB e Python rappresentino due ecosistemi di sviluppo completamente differenti, il primo **proprietario** e il secondo **open source**, l'elaborazione vera e propria per la fattorizzazione di Cholesky su matrici sparse e definite positive è affidata al medesimo motore computazionale: la libreria **CHOLMOD**.

3.1 Il Core Algoritmico: CHOLMOD

CHOLMOD[1] è il cuore tecnologico utilizzato sia da MATLAB che da Python per risolvere i sistemi lineari tramite il metodo di **Cholesky**. Si tratta di una libreria scritta in linguaggio C, progettata per garantire la massima velocità di calcolo sfruttando librerie matematiche standard (**LAPACK** e **BLAS**) che ottimizzano l'uso del processore.

Il vantaggio competitivo di CHOLMOD risiede nella sua natura **ibrida**: la libreria è in grado di selezionare automaticamente tra un metodo **supernodale** (che sfrutta al massimo la cache di memoria tramite chiamate BLAS ed è ideale per matrici con pattern di densità specifici) e un metodo non-supernodale **up-looking** (molto più rapido per matrici estremamente sparse). La selezione avviene durante la fase di analisi simbolica, basandosi sul rapporto tra il numero di operazioni in virgola mobile previste e il numero di elementi non nulli.

Infatti la libreria non applica in modo statico lo stesso algoritmo, ma adatta la propria strategia computazionale in base alle caratteristiche strutturali e alla densità della matrice sparsa in ingresso.

3.2 Integrazione nell'ambiente MATLAB

Nel software proprietario MATLAB, la libreria CHOLMOD è integrata in modo nativo e risulta opaca all'utente. L'accesso a questo motore di calcolo avviene tramite l'operatore predefinito per la risoluzione dei sistemi lineari (il simbolo `\`), che sotto il cofano invoca la funzione `mldivide`[2].

È fondamentale notare l'ottimizzazione di questo processo: quando `mldivide` riceve in input un sistema $Ax = b$ e rileva che la matrice A è **sparsa**, non procede con un calcolo cieco, ma attiva un algoritmo diagnostico dedicato. Seguendo lo specifico albero decisionale (immagine 3.1) per le matrici sparse, il percorso logico che conduce all'adozione del solutore di Cholesky (e quindi all'uso di CHOLMOD) attraversa i seguenti passaggi:

1. Verifica della dimensionalità: Il sistema controlla innanzitutto se la matrice A è **quadrata**. In caso negativo, devia su un solutore basato su fattorizzazione QR.
2. Analisi della banda: Se A è quadrata, ne calcola la larghezza di banda (bandwidth) e verifica che non rientri in casi **banali** o **pre-ottimizzati**: scarta l'ipotesi che sia puramente diagonale,

triangolare, tridiagonale o con una densità di banda superiore alla soglia predefinita (di default 0.5), e si assicura che non sia una matrice triangolare permutata.

3. Controllo di simmetria: Superati i filtri strutturali di base, l'algoritmo verifica se A è **Hermitiana** (ovvero simmetrica, assumendo valori reali).
4. Analisi della diagonale: Se la matrice è Hermitiana, viene effettuato un rapido controllo per verificare che gli elementi sulla diagonale principale siano tutti **positivi** (o tutti **negativi**).
5. Tentativo di fattorizzazione: Soddisfatte queste condizioni, `mldivide` lancia la fattorizzazione di **Cholesky**. Se l'operazione ha successo, confermando intrinsecamente che la matrice è simmetrica e definita positiva, MATLAB adotta il Cholesky solver basato sulle routine ad alte prestazioni di CHOLMOD. (Se invece dovesse fallire, il sistema attuerebbe un fallback su un solutore basato su fattorizzazione LDL^T se A reale o LU altrimenti).

Questa architettura ad albero decisionale garantisce che l'invocazione di CHOLMOD in MATLAB avvenga solo quando le caratteristiche topologiche e numeriche della matrice garantiscono la massima efficienza del metodo.

3.3 Implementazione in ambiente Python: scikit-sparse

Nel panorama open source di Python, l'accesso a questo potente motore di calcolo è reso possibile dalla libreria **scikit-sparse**. Questo modulo funge da **wrapper** avanzato: fa da ponte tra l'ambiente ad alto livello di Python e la libreria precompilata CHOLMOD in C. Utilizzando **scikit-sparse**, lo sviluppatore Python è in grado di configurare, invocare e gestire le stesse esatte routine di fattorizzazione supernodale a cui ha accesso MATLAB.

3.3.1 Gestione dei Dati in Python

Nel panorama open source di Python, l'accesso alle funzionalità di calcolo ad alte prestazioni della libreria CHOLMOD è reso possibile dal wrapper avanzato scikit-sparse. Per l'acquisizione e la gestione dei dataset estratti dalla *SuiteSparse Matrix Collection*, si è deciso di scaricare e utilizzare le matrici nel formato standard **.mtx** (Matrix Market Exchange Format).

Il formato .mtx

Il formato **.mtx**^[3] è una specifica basata su file in puro testo ASCII, progettata per garantire l'interscambio e la massima portabilità di matrici sparse tra piattaforme e linguaggi differenti. La sua struttura lineare prevede un'intestazione iniziale in cui vengono dichiarate le dimensioni complessive (numero di righe, numero di colonne e conteggio dei valori non nulli), seguita da una sequenza di righe in cui ogni riga descrive un singolo elemento tramite la tripletta composta da indice di riga, indice di colonna e valore numerico associato.

In termini di efficienza di archiviazione, emerge un netto divario rispetto all'ecosistema MATLAB: il formato testuale **.mtx** risulta significativamente più pesante su disco rispetto al formato proprietario **.mat**. Se quest'ultimo memorizza le informazioni in formato binario compresso ottimizzando al massimo lo spazio occupato dai dati in doppia precisione, il formato **.mtx** mappa ogni cifra, esponente o spazio vuoto come un singolo carattere testuale, determinando un incremento volumetrico notevole dei file.

Flusso di parsing

Per rimediare ai limiti del formato testuale e permettere una corretta manipolazione numerica dei dati, la pipeline di caricamento all'interno dello script Python è stata strutturata sfruttando la libreria **SciPy**:

1. **I/O e Parsing dei file**: La lettura del file **.mtx** e la deserializzazione delle coordinate testuali in vettori numerici vengono gestite tramite le funzioni del sottomodulo `scipy.io`.

2. **Strutturazione della matrice sparsa:** Immediatamente dopo la lettura, la matrice viene istanziata nella memoria RAM del sistema tramite il modulo `scipy.sparse`, forzando l’allocazione sotto forma di `csc_matrix` (Compressed Sparse Column).

L’adozione specifica del formato `csc_matrix` costituisce un vincolo ingegneristico fondamentale per il progetto. La rappresentazione sparsa compressa per colonne rispecchia in modo speculare la struttura dati `cholmod_sparse` richiesta nativamente dal motore in C di **CHOLMOD**. Questo accorgimento assicura che, una volta superata la fase di lettura da disco, la matrice risieda in memoria RAM in un formato binario ottimizzato, eliminando l’overhead di conversione nel passaggio dei puntatori al wrapper `scikit-sparse` e garantendo una parità di efficienza strutturale rispetto alla gestione interna operata da MATLAB.

3.4 Implicazioni per il Benchmarking

Questa convergenza tecnologica ha un’implicazione profonda per la presente analisi. Poiché l’algoritmo matematico centrale è identico in entrambi i casi, le eventuali discrepanze prestazionali (in termini di tempi di esecuzione e occupazione di memoria RAM) che emergeranno dai test non saranno imputabili a differenze matematiche, bensì a fattori sistemistici e architetturali.

3.5 Considerazioni su Usabilità e Documentazione

Un riflesso diretto della condivisione del motore **CHOLMOD** riguarda l’esperienza d’uso e il reperimento delle informazioni tecniche. Sebbene MATLAB e Python presentino sintassi differenti, la logica operativa per la risoluzione dei sistemi simmetrici definiti positivi è pressoché identica. Poiché entrambe le interfacce espongono le funzionalità della **medesima** libreria C, i parametri fondamentali, come le soglie per il passaggio tra metodo supernodale e non-supernodale o le opzioni di pre-ordinamento (AMD, METIS), rimangono costanti. Questo rende il passaggio da un ambiente all’altro estremamente intuitivo per un utente tecnico.

La documentazione di riferimento per entrambi i sistemi converge in ultima analisi verso la guida ufficiale di **CHOLMOD**. Le specifiche fornite da MathWorks per `chol` e la documentazione di `scikit-sparse` per Python descrivono gli stessi comportamenti algoritmici, le stesse statistiche di output e gli stessi vincoli di memoria.

In conclusione, la scelta tra **MATLAB** e **Python** non è influenzata da una differenza nella “qualità” delle informazioni disponibili o nella complessità logica del comando, poiché entrambi gli ambienti offrono un’astrazione di alto livello sopra lo stesso, solido standard industriale.

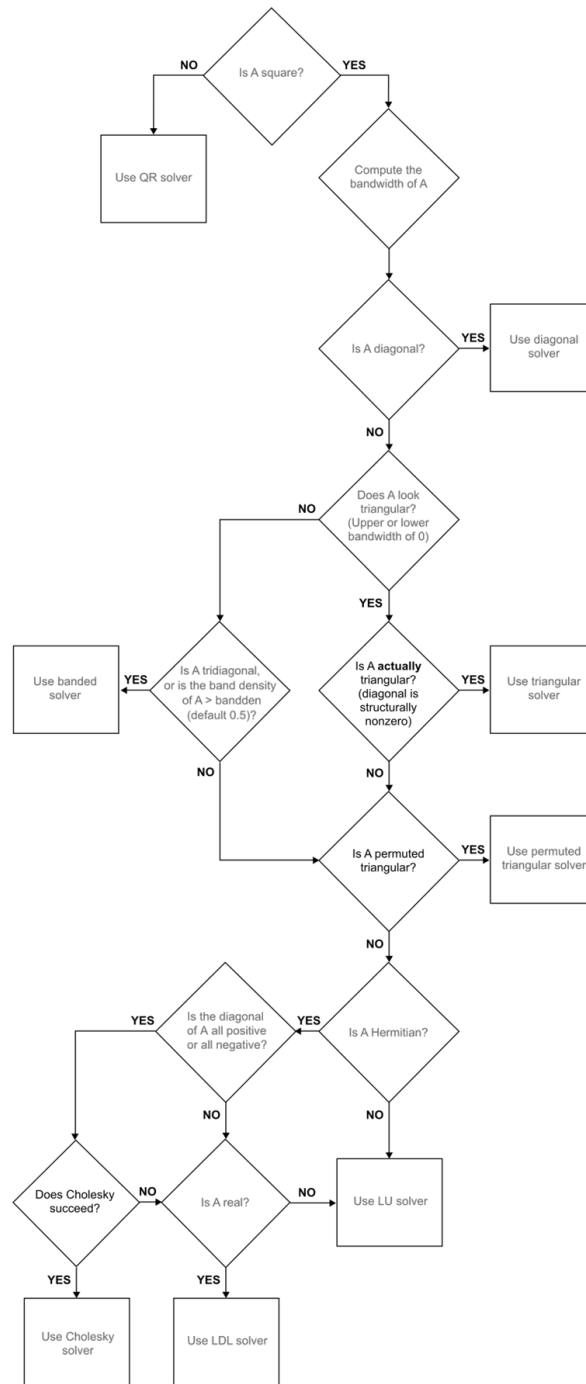


Figure 3.1: Algoritmo per le matrici sparse [2]

Chapter 4

Risultati Sperimentali e Analisi dei Benchmark

In questo capitolo vengono presentati e analizzati i risultati quantitativi emersi dalla campagna di test. Di seguito verranno illustrati i grafici riassuntivi elaborati a partire dalle misurazioni effettuate sul dataset di matrici sparse precedentemente descritte, tracciando le performance algoritmiche al crescere della dimensione dei dati in ingresso.

L'obiettivo di questa sezione è fornire un confronto visivo e immediato delle prestazioni registrate nei due ecosistemi di calcolo (**MATLAB** e **Python**) eseguiti sui due differenti sistemi operativi (**Windows** e **Linux**). Per garantire una lettura coerente, l'esposizione dei dati si articola sui tre indicatori chiave definiti nella metodologia:

- **Velocità di elaborazione:** Analisi dei tempi di esecuzione.
- **Accuratezza numerica:** Valutazione della stabilità tramite il calcolo dell'errore relativo.
- **Impatto infrastrutturale:** Misurazione del picco di occupazione della memoria RAM.

Per favorire una corretta interpretazione visiva, si fa notare che in tutti i grafici proposti è stata adottata una scala **logaritmica** sull'asse delle ordinate. Questa scelta rappresentativa si è resa strettamente necessaria poiché i valori rilevati, sia in termini di tempi di esecuzione che di consumo di memoria o di tolleranza di errore, coprono ordini di grandezza profondamente diversi al variare della complessità del sistema lineare analizzato.

4.1 Performance su Windows

4.1.1 MATLAB

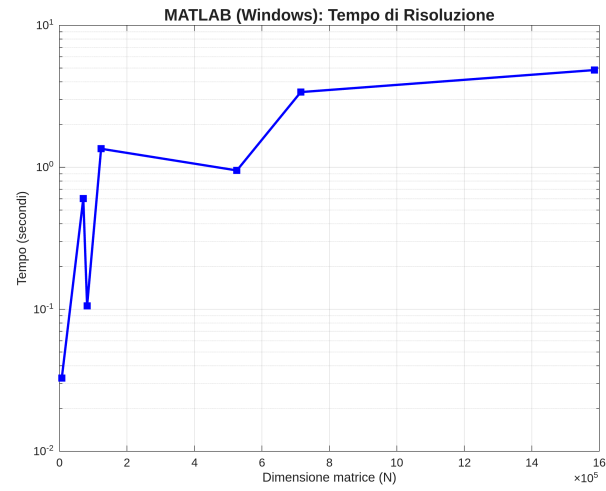


Figure 4.1: Tempo di risoluzione di MATLAB su Windows

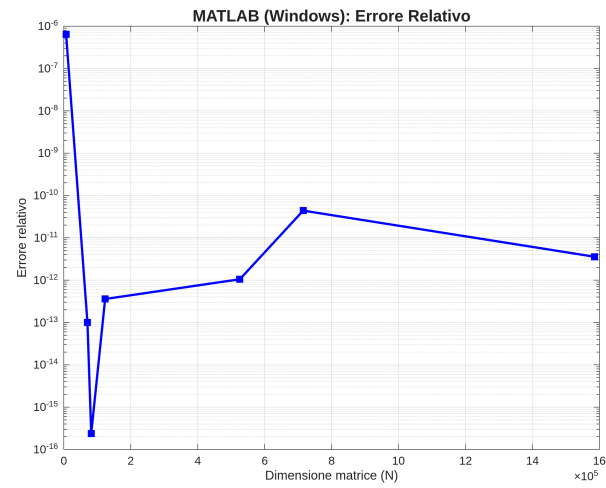


Figure 4.2: Errore relativo di MATLAB su Windows

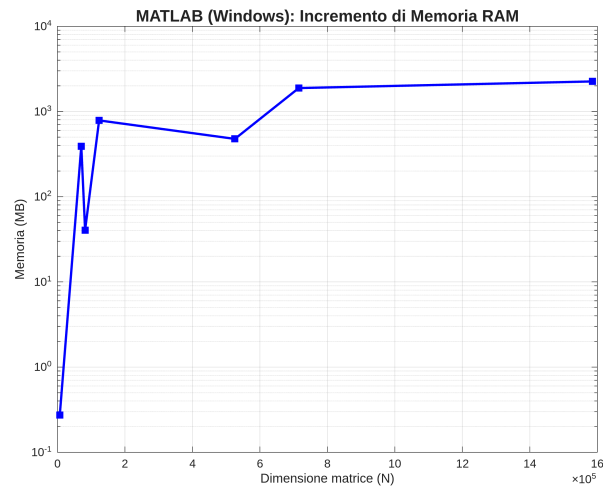


Figure 4.3: Memoria peak di MATLAB su Windows

Matrice	Dimensione (N)	Tempo (s)	Errore relativo	Memoria (MB)
ex15	6867	0.0327	6.386e-07	0.2734
cf1	70656	0.6018	9.954e-14	391.5117
shallow_water1	81920	0.1057	2.377e-16	40.5469
cf2	123440	1.3501	3.591e-13	785.6758
parabolic_fem	525825	0.9501	1.05e-12	476.6445
apache2	715176	3.3845	4.393e-11	1882.332
G3_circuit	1585478	4.8382	3.575e-12	2252.1719

Table 4.1: Dati della risoluzione di MATLAB su Windows

4.1.2 Python

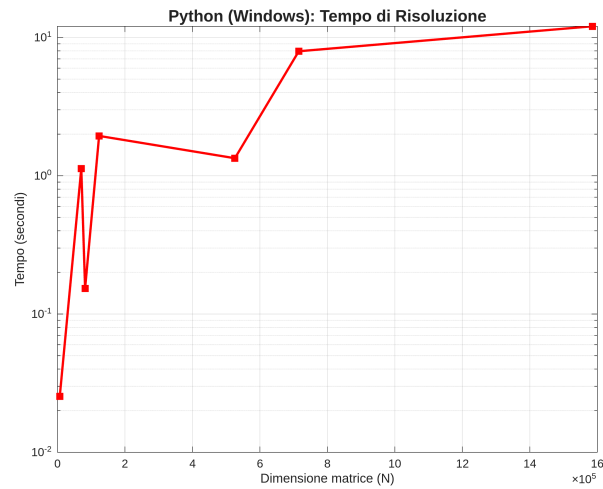


Figure 4.4: Tempo di risoluzione di Python su Windows

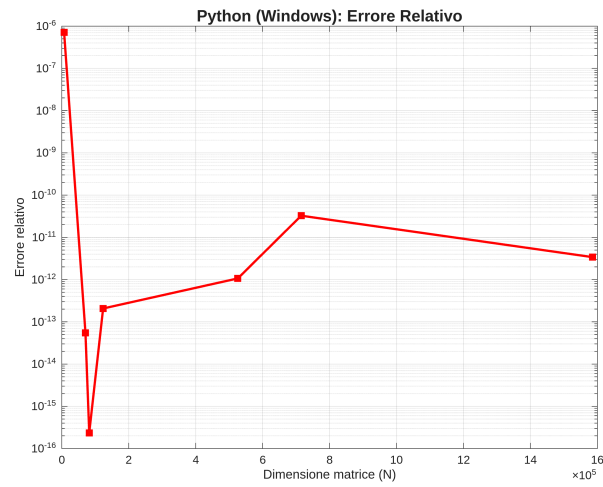


Figure 4.5: Errore relativo di Python su Windows

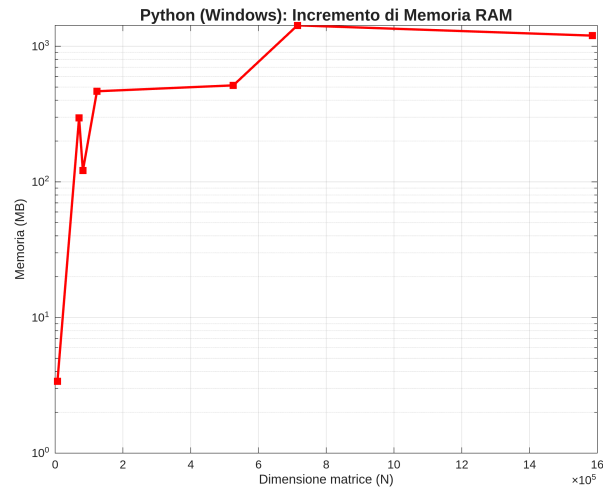


Figure 4.6: Memoria peak di Python su Windows

Matrice	Dimensione (N)	Tempo (s)	Errore relativo	Memoria (MB)
ex15	6867	0.0254	7.1027e-07	3.3867
cf1	70656	1.1258	5.4843e-14	296.5508
shallow_water1	81920	0.1533	2.3577e-16	121.0234
cf2	123440	1.9414	2.0697e-13	465.5781
parabolic_fem	525825	1.3390	1.0659e-12	514.7383
apache2	715176	7.9519	3.2537e-11	1425.6562
G3_circuit	1585478	12.0525	3.4006e-12	1197.7031

Table 4.2: Dati della risoluzione di Python su Windows

4.2 Performance su Linux

4.2.1 MATLAB

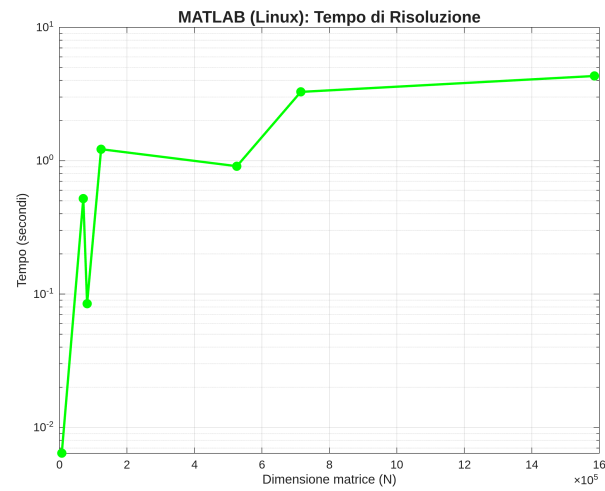


Figure 4.7: Tempo di risoluzione di MATLAB su Linux

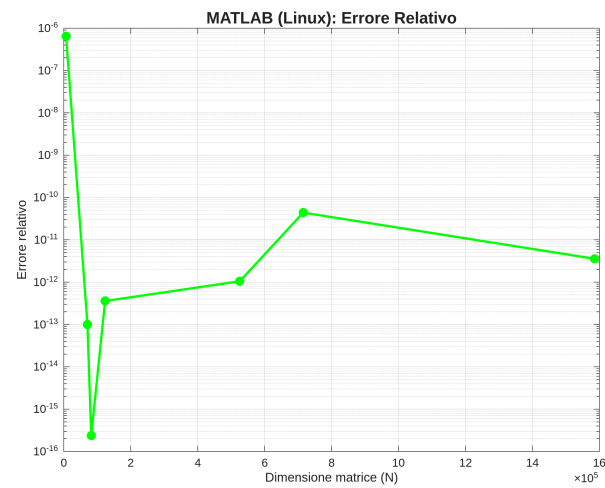


Figure 4.8: Errore relativo di MATLAB su Linux

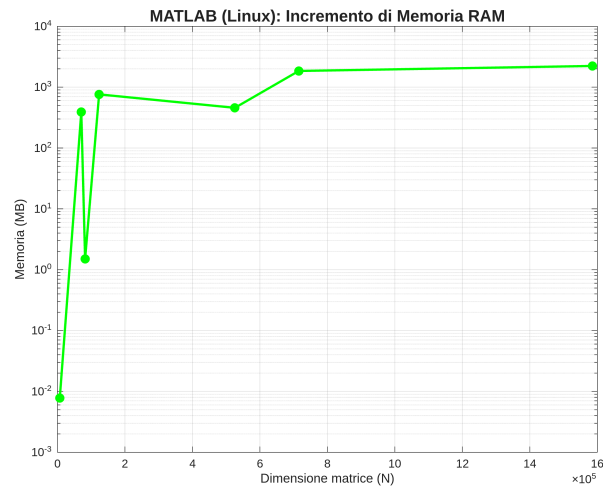


Figure 4.9: Memoria peak di MATLAB su Linux

Matrice	Dimensione (N)	Tempo (s)	Errore relativo	Memoria (MB)
ex15	6867	0.0064	6.386e-07	0.0078
cf1	70656	0.5207	9.954e-14	389.3867
shallow_water1	81920	0.0848	2.377e-16	1.5078
cf2	123440	1.2185	3.591e-13	760.3555
parabolic_fem	525825	0.9092	1.05e-12	457.3281
apache2	715176	3.2774	4.393e-11	1844.5156
G3_circuit	1585478	4.3137	3.575e-12	2227.7344

Table 4.3: Dati della risoluzione di MATLAB su Linux

4.2.2 Python

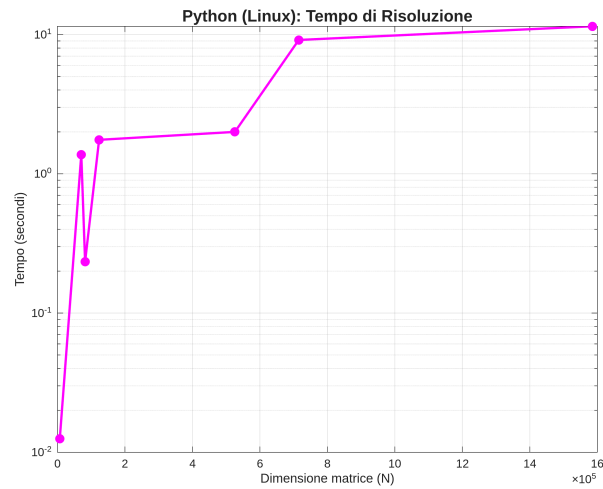


Figure 4.10: Tempo di risoluzione di Python su Linux

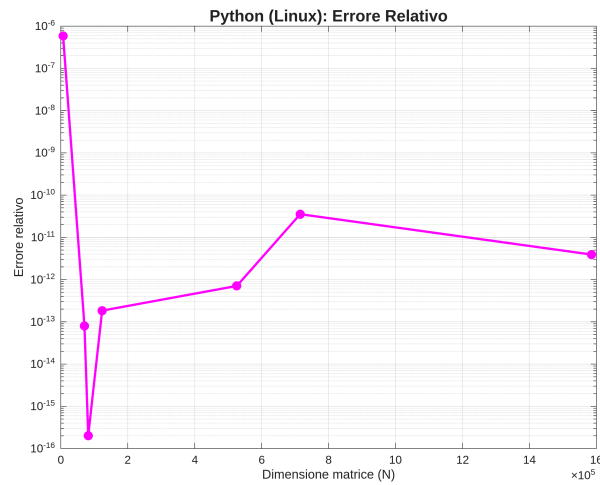


Figure 4.11: Errore relativo di Python su Linux

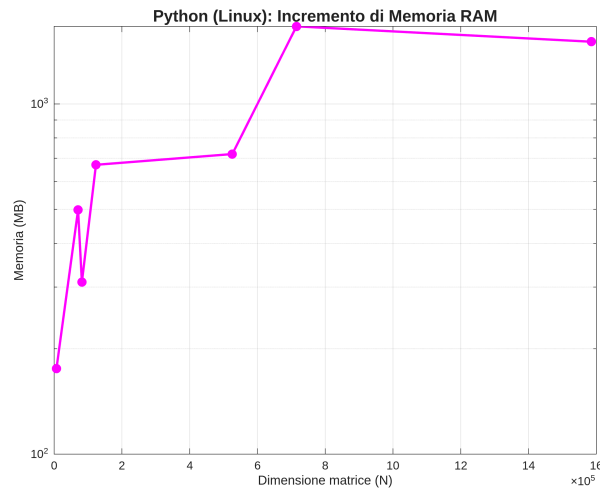


Figure 4.12: Memoria peak di Python su Linux

Matrice	Dimensione (N)	Tempo (s)	Errore relativo	Memoria (MB)
ex15	6867	0.0125	5.8895e-07	175.3477
cf1	70656	1.3751	8.0466e-14	498.2188
shallow_water1	81920	0.2336	2.0162e-16	309.8750
cf2	123440	1.7537	1.8410e-13	670.3711
parabolic_fem	525825	1.9999	7.0730e-13	719.4180
apache2	715176	9.1467	3.5369e-11	1666.1641
G3_circuit	1585478	11.4687	3.9157e-12	1506.0391

Table 4.4: Dati della risoluzione di Python su Linux

4.3 Analisi Comparativa

4.3.1 MATLAB: Windows vs Linux

L'osservazione dei grafici prestazionali restituisce un quadro estremamente **coerente** riguardo l'esecuzione dell'ambiente proprietario **MATLAB** sui due differenti sistemi operativi. I test, eseguiti in **condizioni hardware identiche**, permettono di isolare con precisione l'impatto dell'infrastruttura software sull'efficienza computazionale del motore di risoluzione.

Andamento dei Tempi di risoluzione

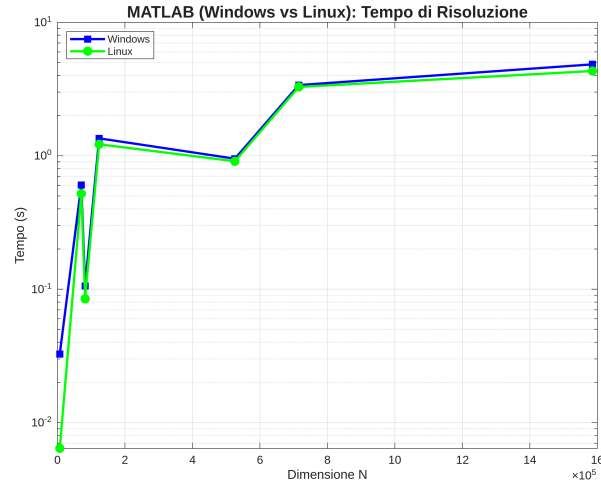


Figure 4.13: Confronto del tempo di risoluzione di MATLAB sui due sistemi operativi

L'analisi del grafico relativo ai tempi di esecuzione evidenzia una forte correlazione tra le due architetture, delineando al contempo un leggero ma costante vantaggio per l'ambiente **Linux**. Al crescere della dimensione del sistema lineare, i tempi di calcolo aumentano seguendo una traiettoria quasi **identica**. Tuttavia, esaminando nel dettaglio le singole misurazioni, si evince che la curva prestazionale associata a Linux si colloca sistematicamente al di sotto di quella di Windows. Questo divario, seppur frazionario, si mantiene stabile su tutte le scale di grandezza analizzate, dalle matrici di ordine inferiore fino a quelle composte da milioni di elementi. Tale comportamento strutturale suggerisce che il kernel **Linux** gestisca in modo più efficiente la schedulazione dei processi e l'overhead di sistema, traducendosi in un tempo netto di calcolo inferiore pur impiegando gli stessi algoritmi risolutivi.

Accuratezza numerica ed errore relativo

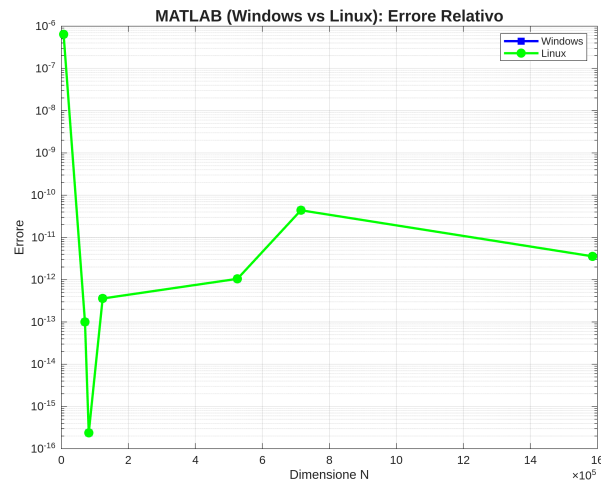


Figure 4.14: Confronto dell'errore relativo di MATLAB sui due sistemi operativi

Spostando l'attenzione sulla stabilità dei calcoli, il grafico dell'errore relativo conferma la **totale ed esatta** parità matematica tra le due implementazioni. Le curve corrispondenti alle misurazioni su Windows e Linux risultano perfettamente **sovrapposte** lungo tutto lo spettro dei test. Questo riscontro empirico corrobora la solidità teorica dell'impianto di benchmarking: essendo il calcolo vettoriale delegato in ultima istanza alla **medesima** libreria precompilata condivisa (CHOLMOD), le operazioni in virgola mobile e i fisiologici errori di arrotondamento vengono replicati in modo millimetrico. Il

sistema operativo ospite non introduce, pertanto, alcuna **perturbazione** nella precisione numerica del risultato finale.

Dinamiche di occupazione della memoria RAM

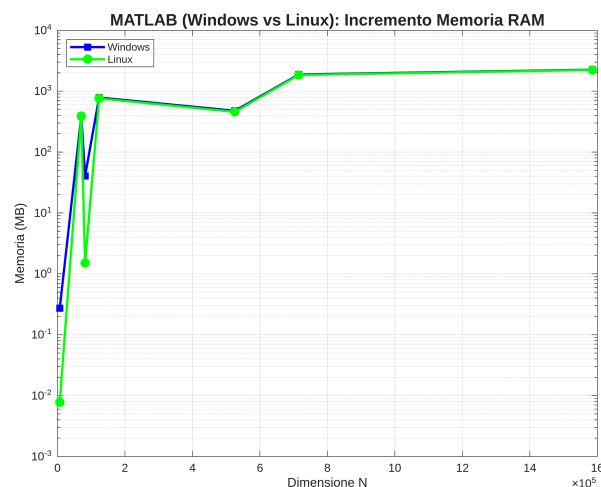


Figure 4.15: Confronto della memoria peak di MATLAB sui due sistemi operativi

La misurazione dell'incremento di memoria RAM offre prospettive interessanti sulla gestione iniziale delle risorse e sull'allocazione dei processi. Per le matrici di dimensioni contenute emerge un distacco marcato a favore dell'ambiente **Linux**, il quale registra un assorbimento di memoria iniziale di quasi un **ordine di grandezza** inferiore rispetto a Windows. Questo divario iniziale indica che l'ecosistema **Windows** richiede l'allocazione di un volume maggiore di memoria di base per l'inizializzazione del solutore e il caricamento del framework MATLAB. Al contrario, man mano che la mole dei dati elaborati cresce, l'**ingombro strutturale** della matrice diviene il fattore dominante, rendendo del tutto marginale l'**overhead** iniziale di sistema. Di conseguenza, per i carichi di lavoro più gravosi, le curve di assorbimento convergono totalmente, dimostrando che il peso netto della fattorizzazione sulla RAM risulta equivalente su entrambe le piattaforme.

4.3.2 Python: Windows vs Linux

L'esame dei dati raccolti durante le sessioni di test basate sull'ecosistema open source di Python evidenzia dinamiche d'uso interessanti, tracciando un comportamento parzialmente **divergente** rispetto a quanto riscontrato con il software proprietario. Anche in questo scenario, l'adozione del medesimo algoritmo di calcolo permette di mappare con precisione le interazioni tra l'interprete del linguaggio, i wrapper di interfacciamento e il sistema operativo ospite.

Andamento dei Tempi di risoluzione

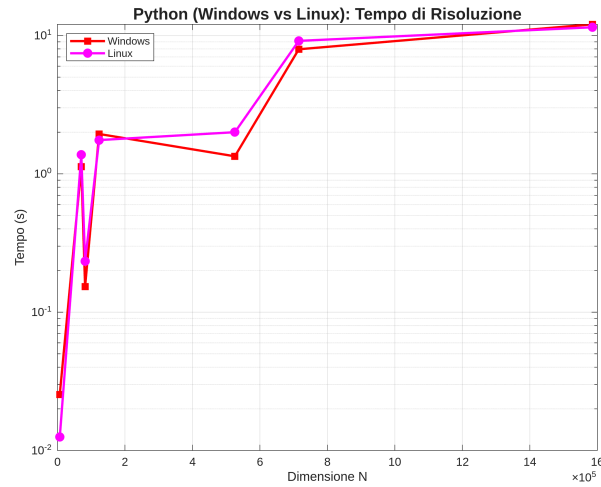


Figure 4.16: Confronto del tempo di risoluzione di Python sui due sistemi operativi

Il grafico relativo ai tempi di calcolo di Python mostra un andamento inizialmente **speculare** tra le due piattaforme, che tende però a differenziarsi in modo netto all'**aumentare** della complessità del problema. Nelle matrici di dimensioni ridotte, Linux e Windows si rincorrono con scarti infinitesimali. Tuttavia, superata la soglia di dimensioni intermedie, si osserva un comportamento **controintuitivo**: la curva di Windows si attesta stabilmente al di sotto di quella di Linux, evidenziando tempi di esecuzione più rapidi sul sistema operativo di Microsoft. Questa inversione prestazionale suggerisce che, nei carichi di lavoro più **intensivi**, l'estensione precompilata utilizzata da Python o le librerie BLAS/LAPACK collegate nell'ambiente Windows riescano a sfruttare in modo più efficiente il multithreading o l'ottimizzazione dei registri della CPU rispetto alla configurazione analizzata su Linux, compensando e superando la naturale snellezza del kernel di quest'ultimo.

Accuratezza numerica ed errore relativo

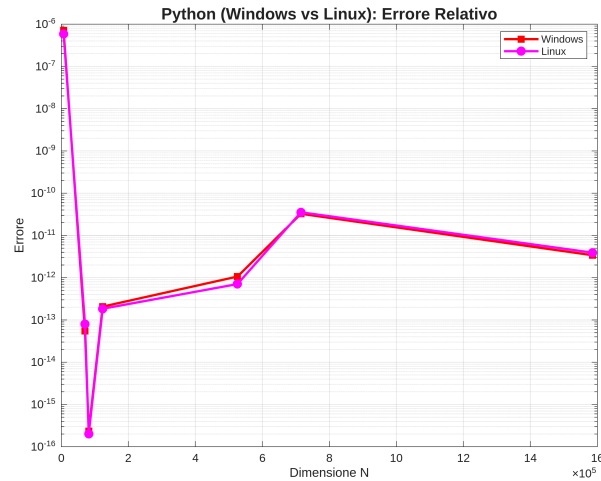


Figure 4.17: Confronto dell'errore relativo di Python sui due sistemi operativi

Il profilo dell'accuratezza numerica si conferma un indicatore puramente dipendente dall'algoritmo e del tutto **immune** dalle variazioni sistemiche. Le curve dell'errore relativo registrate su Windows e su Linux risultano virtualmente **sovrapposte** per l'intero campionamento di matrici sparse. Salvo oscillazioni microscopiche e fisiologiche dovute a minime variazioni nell'**arrotondamento** dei float a livello di compilatore hardware, l'accuratezza numerica segue la medesima traiettoria su entrambi

i sistemi operativi. Questo risultato attesta la perfetta affidabilità del wrapper `scikit-sparse` nel trasferire i dati in doppia precisione alla libreria C sottostante senza introdurre alcuna degradazione del dato.

Dinamiche di occupazione della memoria RAM

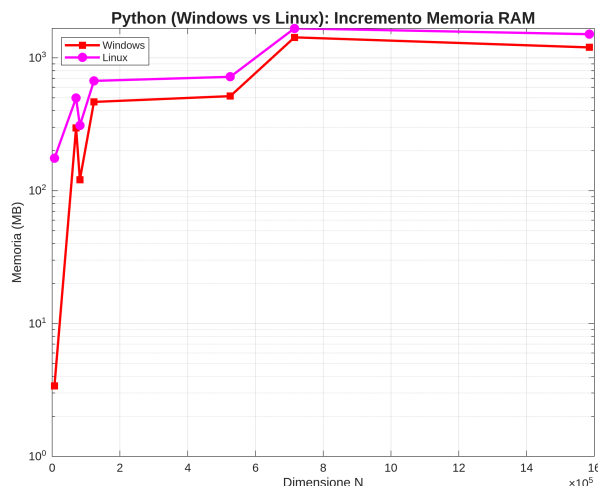


Figure 4.18: Confronto della memoria peak di Python sui due sistemi operativi

La misurazione del picco di memoria RAM consumata esibisce il divario più **marcato** tra i due ambienti operativi, delineando una chiara gerarchia di efficienza. Contro ogni aspettativa basata sui benchmark generali, la curva di Windows si posiziona costantemente e nettamente al di sotto di quella di Linux per tutta la durata del test.

Questa discrepanza sistematica è riconducibile a due possibili fattori **principali**:

- **Strategia di allocazione della memoria:** Il gestore della memoria di Windows (Heap Manager) interviene in modo molto **aggressivo** nel compattare la memoria heap e riorganizzare le pagine non appena Python distrugge le matrici temporanee usate per la risoluzione del sistema lineare. Al contrario, l'allocatore standard di Linux (`glibc malloc`) adotta una strategia più **conservativa**, tendendo a trattenere temporaneamente le risorse liberate dal processo per destinarle a calcoli futuri, il che fa lievitare **artificialmente** il picco di RAM registrato.
- **Ottimizzazione delle librerie compilate:** I pacchetti precompilati di **OpenBLAS** distribuiti per l'ambiente Windows beneficiano spesso di catene di compilazione commerciali altamente ottimizzate per i registri delle CPU desktop standard. Questa stretta sinergia riduce l'accumulo di strutture dati **ridondanti** in memoria durante il passaggio dei puntatori al wrapper per `scikit-sparse`, garantendo un'impronta strutturale decisamente più leggera rispetto alla configurazione standard utilizzata su Linux.

4.3.3 Windows: MATLAB vs Python

Il terzo scenario di test si concentra sul confronto diretto tra i due linguaggi di programmazione, operando a parità di sistema operativo, Windows. Questa specifica prospettiva analitica permette di valutare l'**efficienza** dei rispettivi ecosistemi software nell'interfacciarsi con il medesimo motore di calcolo sottostante.

Andamento dei Tempi di risoluzione

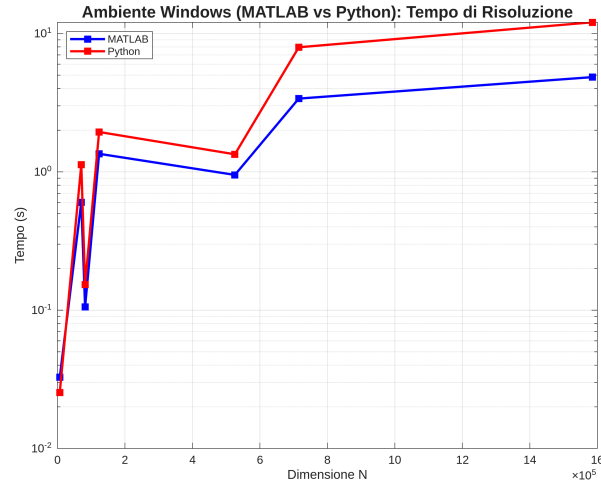


Figure 4.19: Confronto del tempo di risoluzione dei due linguaggi su Windows

L'analisi del grafico relativo ai tempi di risoluzione fa emergere una netta e costante supremazia prestazionale da parte di **MATLAB**. Sebbene per matrici di dimensioni microscopiche le prestazioni appaiano quasi sovrapponibili, al crescere della complessità del sistema lineare la curva di MATLAB si distacca, posizionandosi stabilmente e visibilmente al di sotto di quella di Python. Questo divario temporale, che si consolida e si accentua sui carichi di lavoro più gravosi, evidenzia il vantaggio architettonico derivante dalla profonda integrazione nativa di CHOLMOD all'interno dell'ecosistema chiuso di MATLAB. L'ambiente Python, pur vantando librerie matematiche altamente ottimizzate, sconta l'inevitabile overhead introdotto dal wrapper `scikit-sparse` e dal passaggio di informazioni tra il livello interpretato e la libreria precompilata in C, traducendosi in un tempo di elaborazione fisiologicamente superiore.

Accuratezza numerica ed errore relativo

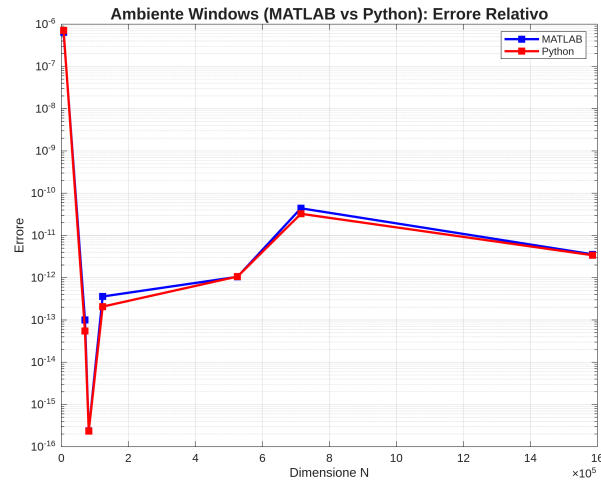


Figure 4.20: Confronto dell'errore relativo dei due linguaggi su Windows

Spostando il focus sulla stabilità numerica trans-linguaggio, il grafico dell'errore relativo ribadisce la forte **convergenza** matematica delle due soluzioni. Le curve di MATLAB e Python risultano strettamente allineate lungo l'intera **progressione** delle dimensioni matriciali. Questo risultato empirico è la logica e diretta conseguenza dell'architettura condivisa illustrata nei capitoli precedenti: entrambi gli ambienti, pur operando tramite interfacce ad alto livello profondamente diverse, delegano il nucleo

del calcolo algoritmico alla medesima libreria sottostante scritta in C. Poiché le operazioni a doppia precisione vengono eseguite fisicamente dallo stesso motore matematico, l'accuratezza del risultato si mantiene **coerente**, dimostrando che gli eventuali scostamenti sono relegati al rumore numerico di fondo e che il linguaggio "contenitore" non introduce alcuna degradazione sistematica dell'informazione elaborata.

Dinamiche di occupazione della memoria RAM

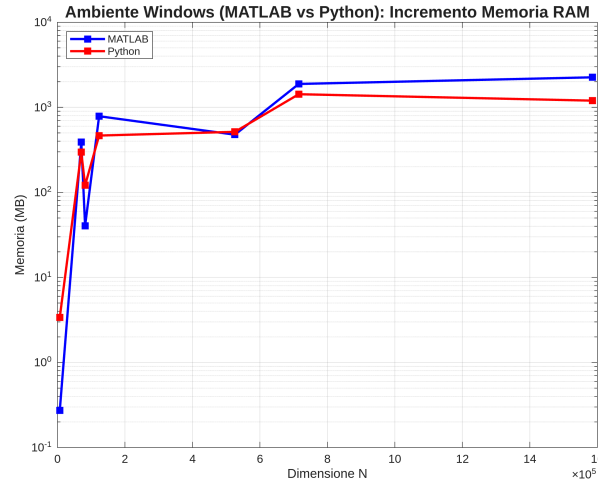


Figure 4.21: Confronto della memoria peak dei due linguaggi su Windows

L'aspetto più **controintuitivo** e interessante del confronto su piattaforma Windows emerge dall'analisi dell'incremento di memoria RAM. In questo specifico ambito, l'ecosistema open source di Python dimostra un'**efficienza** strutturale superiore rispetto alla controparte proprietaria. La curva associata a Python si colloca costantemente al di sotto di quella di MATLAB, indicando un'impronta di memoria sistematicamente più snella durante l'intera fase di elaborazione. Questo riscontro empirico suggerisce che il framework completo di MATLAB, per supportare la sua massima velocità di esecuzione, richieda l'allocazione di un volume di risorse accessorie **maggiore**. Al contrario, la pipeline composta da **scikit-sparse** si rivela più parsimoniosa, liberando le risorse o limitando l'ingombro strutturale in modo più efficace una volta che i dati sono stati allocati per la fattorizzazione.

4.3.4 Linux: MATLAB vs Python

L'ultimo scenario di benchmarking chiude il cerchio delle osservazioni confrontando le prestazioni dei due linguaggi di programmazione all'interno dell'ambiente open source fornito da Linux. Questa configurazione è particolarmente significativa, in quanto permette di valutare come i due ecosistemi interagiscano con un kernel storicamente ottimizzato per i carichi di lavoro computazionali e la gestione avanzata dei processi.

Andamento dei Tempi di risoluzione

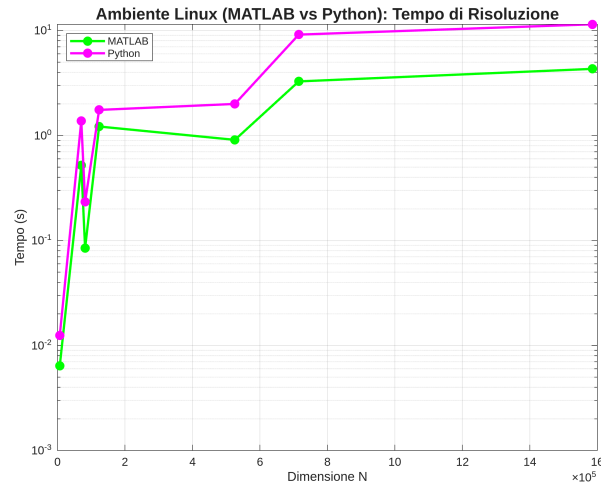


Figure 4.22: Confronto del tempo di risoluzione dei due linguaggi su Linux

L'analisi del grafico relativo ai tempi di esecuzione restituisce un verdetto **inequivocabile**: su piattaforma Linux, la superiorità prestazionale di **MATLAB** rispetto a Python risulta ancora più pronunciata di quanto osservato su Windows. La curva di MATLAB si posiziona stabilmente e in modo netto al di sotto della curva di Python per l'intero set di test. Questa divergenza si amplifica progressivamente all'aumentare della dimensione N della matrice. Tale comportamento evidenzia come l'integrazione nativa del motore **CHOLMOD** all'interno di MATLAB riesca a capitalizzare al massimo l'efficienza dello **scheduler** di Linux. Di contro, l'ambiente Python sembra soffrire in misura maggiore l'overhead legato al wrapping delle funzioni e, potenzialmente, una minore ottimizzazione delle librerie precompilate (come OpenBLAS) fornite di default sulla distribuzione utilizzata, traducendosi in tempi operativi sensibilmente più dilatati per i calcoli su larga scala.

Accuratezza numerica ed errore relativo

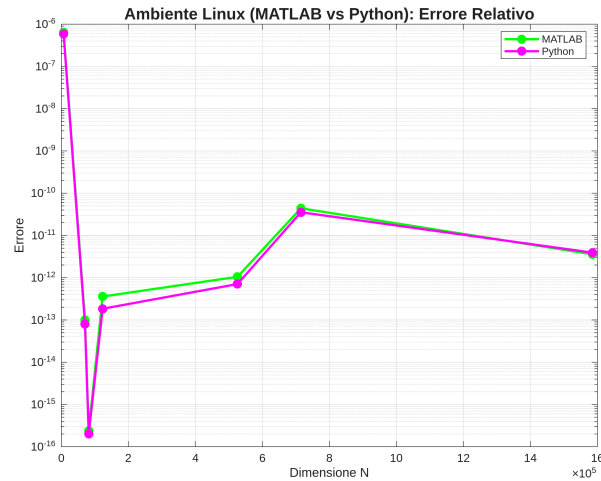


Figure 4.23: Confronto dell'errore relativo dei due linguaggi su Linux

In linea con le rilevazioni precedenti, il grafico dell'errore relativo conferma una **sostanziale** equivalenza matematica tra i due linguaggi. Le misurazioni dell'accuratezza seguono il **medesimo** andamento lungo tutto l'asse dimensionale, sovrapponendosi entro i fisiologici limiti della **tolleranza** di macchina. Questo ennesimo riscontro consolida la tesi fondamentale del progetto: l'affidabilità numerica della risoluzione dei sistemi lineari sparsi è **garantita** dal nucleo algoritmico in C. Le discrepanze

microscopiche riscontrabili tra le curve non rappresentano una degradazione del dato, ma sono imputabili ai normali errori di **arrotondamento** del calcolo in virgola mobile e alle minime **differenze** nelle librerie di algebra lineare di base (BLAS/LAPACK) richiamate dai due ecosistemi per orchestrare l'esecuzione.

Dinamiche di occupazione della memoria RAM

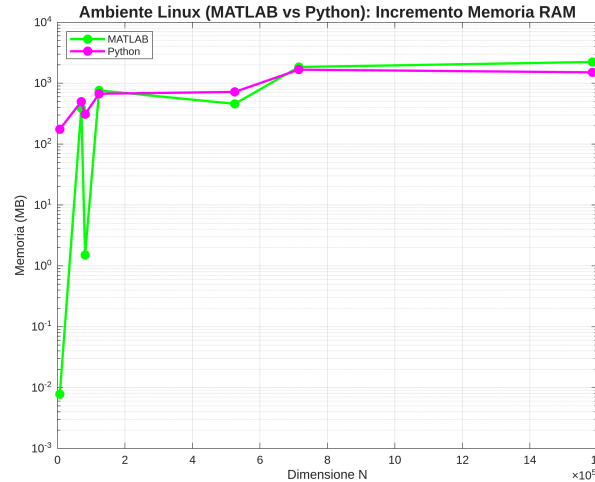


Figure 4.24: Confronto della memoria peak dei due linguaggi su Linux

L'esame dell'incremento del picco di memoria RAM, monitorato attraverso il parametro del *Resident Set Size* (RSS), offre uno spaccato fondamentale sulle differenti filosofie di gestione delle risorse dei due ambienti. Poiché il protocollo di benchmark prevede rigorosamente la misurazione dell'assorbimento netto nell'intervallo che intercorre dall'istante di **caricamento** della matrice fino al **completamento** della risoluzione, i grafici evidenziano una discrepanza sistematica, condizionata dalle architetture software dei due framework.

Per i carichi di lavoro ridotti, l'incremento di memoria registrato da MATLAB risulta **microscopico**, attestandosi su valori stabili inferiori allo 0.02 MB. Tale comportamento è la diretta conseguenza della strategia di allocazione monolitica del software proprietario. All'avvio dell'applicazione, durante la fase di caricamento iniziale, MATLAB pre-alloca massicce aree di memoria virtuale (*memory arenas*) dal sistema operativo e inizializza l'intera infrastruttura (inclusi i thread di calcolo parallelo). Quando lo script avvia la risoluzione del sistema, MATLAB non deve richiedere nuove risorse al kernel, ma sfrutta lo spazio virtuale già prenotato. Il sistema operativo interviene esclusivamente tramite il meccanismo del **Demand Paging**, allocando la RAM fisica solo nel momento esatto in cui i dati vengono scritti. Di conseguenza, il costo infrastrutturale di MATLAB rimane totalmente **escluso** dalla finestra temporale del benchmark.

Al contrario, l'ecosistema open source di Python adotta per design una filosofia di **Lazy Initialization**. All'avvio, l'interprete mantiene un'impronta in RAM estremamente leggera. Tuttavia, nell'istante esatto in cui viene invocata la routine di calcolo, la reazione a catena che attiva il wrapper **scikit-sparse**, il motore *CHOLMOD* e la libreria *OpenBLAS* avviene in tempo reale. Questo processo costringe l'allocatore di sistema **glibc malloc** a richiedere istantaneamente al kernel la creazione di blocchi di memoria anonima e la generazione dei *thread pool* necessari per l'esecuzione parallela. Poiché questa inizializzazione infrastrutturale massiccia si sviluppa **interamente** all'interno della finestra di misurazione dell'algoritmo, essa si riflette nei grafici sotto forma del picco di RAM artificialmente **elevato**, quantificabile in diverse decine di megabyte anche per le matrici più piccole.

All'aumentare delle dimensioni del sistema lineare si osserva una netta convergenza strutturale tra le due curve. Sui carichi di lavoro più gravosi, il volume di memoria fisica richiesto per ospitare la struttura della matrice sparsa e il *fill-in* generato dalla fattorizzazione diventa l'elemento dominante. In questa fase di regime, l'overhead di accensione iniziale dell'infrastruttura di Python diventa del tutto marginale rispetto ai gigabyte necessari all'elaborazione numerica. Di conseguenza, l'efficienza

dei due ambienti si allinea, mostrando per le matrici di grandi dimensioni un comportamento asintotico **sovrapponibile**.

Chapter 5

Conclusioni

La vasta campagna di test condotta incrociando i due ambienti di calcolo (**MATLAB** e **Python**) con i due sistemi operativi target (**Windows** e **Linux**) permette di tracciare un quadro esaustivo e non lineare delle prestazioni dei solutori basati su **CHOLMOD**. L'interazione tra i linguaggi ad alto livello, i rispettivi gestori della memoria e le peculiarità dei kernel dei sistemi operativi ha smentito alcuni assunti classici, offrendo solide fondamenta per la definizione di una strategia aziendale e operativa.

5.1 Quadro di sintesi degli andamenti incrociati

L'elemento di maggiore interesse emerso dai benchmark è il comportamento **asimmetrico** dei due linguaggi al variare della piattaforma ospite. **MATLAB** dimostra una naturale sinergia con l'ambiente **Linux**, dove riesce a limare ulteriormente i tempi di calcolo e a ridurre drasticamente l'impatto sulla memoria fisica nelle matrici di piccole dimensioni, grazie a una gestione snella del *Demand Paging* all'interno delle proprie arene virtuali. Al contrario, **Python** manifesta un'inversione prestazionale a favore di **Windows**: sui carichi di lavoro più intensivi, la combinazione su piattaforma Microsoft si è rivelata più rapida ed efficiente dal punto di vista del picco di memoria RAM rispetto alla controparte Linux.

Sotto il profilo dell'**accuratezza numerica**, tutti gli scenari confermano una stabilità equivalente, con errori relativi che si attestano stabilmente nell'intorno del limite inferiore della precisione di macchina (10^{-16}). Questo comportamento uniforme isola l'**efficienza infrastrutturale** e di **gestione della memoria** come gli unici veri fattori discriminanti per la scelta operativa.

5.2 Vincoli architetturali e impatto sulle risorse

La scelta del solutore ideale **non può prescindere** dalla comprensione di come il picco di memoria venga registrato all'interno della finestra temporale dell'algoritmo (dall'istante di caricamento della matrice al calcolo della soluzione).

L'approccio di MATLAB si basa su un'**allocazione monolitica** preventiva. Il software paga il costo infrastrutturale "all'ingresso", occupando un'impronta massiccia di RAM all'avvio per preparare i framework e le librerie. Di conseguenza, il picco misurato durante l'esecuzione dello script riflette quasi esclusivamente il costo matematico puro della fattorizzazione.

Python adotta invece una filosofia di **Lazy Initialization**, mantenendo l'interprete leggero all'avvio ma scaricando l'accensione dell'intera infrastruttura (innesco di **scikit-sparse**, **CHOLMOD** e **OpenBLAS**, allocazione dei *thread pool*) nel mezzo della finestra temporale monitorata. Questo determina un overhead iniziale fisso in RAM di diverse decine di megabyte. Su Linux, tale dinamica è **amplificata** dalla tendenza conservativa dell'allocatore **glibc malloc**, che trattiene la memoria allocata senza ricompattarla immediatamente, innalzando il picco di *Resident Set Size* (RSS). Su Windows, l'*Heap Manager* attua strategie di compattazione e rilascio molto più aggressive, ottimizzando l'uso della RAM sui grandi carichi Python.

5.3 Decisione operativa

Sulla base dei trade-off empirici documentati, la decisione operativa viene strutturata secondo due scenari aziendali distinti, ottimizzando le **risorse hardware** e i **costi di licenza**.

Scenario A: Produzione in Cloud, Cluster High-Performance o Massima Priorità al Time-to-Solution

Qualora l'obiettivo primario sia l'**abbattimento dei tempi** di elaborazione su pipeline industriali **massicce** e si disponga di budget per le **licenze software**, la scelta strategica ricade univocamente su **MATLAB in ambiente Linux**. Questa combinazione offre la massima velocità di esecuzione pura, sfruttando al meglio lo scheduler del kernel Linux e azzerando i colli di bottiglia legati ai wrapper. L'impronta di memoria RAM a regime è interamente dominata dal peso della matrice, garantendo stabilità anche su sistemi lineari vicini al limite fisico della macchina.

Scenario B: Sviluppo Locale Distributed, Vincoli di Budget (Open Source) o Architetture Desktop

Nelle infrastrutture in cui il costo delle licenze rappresenta un **vincolo** o laddove si preferisca l'integrazione nell'ecosistema **open source** dei dati, la scelta d'elezione è **Python in ambiente Windows**. I benchmark dimostrano che, su piattaforma Windows, Python non solo riduce l'ingombro di memoria RAM rispetto a MATLAB su carichi medio-grandi, ma garantisce tempi di risposta inferiori rispetto alla sua stessa controparte Linux. Questa soluzione mitiga l'overhead della *Lazy Initialization* grazie all'efficienza dei pacchetti precompilati per Windows e alla gestione della memoria del sistema operativo, offrendo il miglior compromesso tra costo economico, consumo di risorse hardware e velocità di calcolo.

Chapter 6

Listato del codice

6.1 MATLAB

6.1.1 Script principale

```
1 % Metodi del Calcolo Scientifico - Progetto 1
2 % Hicham Benbouzid 894680
3 % Le Yang Shi 894536
4 % Elena Zen 895355
5
6 clear; clc; close all;
7
8 % Elenco delle matrici
9 matrici_names = {'Flan_1565.mat', ...
10                '%StocF-1465.mat', ...
11                'cfd2.mat', ...
12                'cfd1.mat', ...
13                'G3_circuit.mat', ...
14                'parabolic_fem.mat', ...
15                'apache2.mat', ...
16                'shallow_water1.mat', ...
17                'ex15.mat'};
18
19 num_matrici = length(matrici_names);
20
21 % Inizializzazione degli array per salvare i risultati
22 N_vals = zeros(num_matrici, 1);
23 tempi = zeros(num_matrici, 1);
24 errori = zeros(num_matrici, 1);
25 memorie = zeros(num_matrici, 1);
26
27 fprintf('Inizio decomposizione di Cholesky per tutte le matrici...\n');
28 fprintf('-----\n');
29
30 for i = 1:num_matrici
31     nome_file = fullfile('matrices', matrici_names{i});
32     fprintf("Matrice: %-15s | ", strrep(matrici_names{i}, '.mat', ''));
33
34     % Controlla se il file esiste
35     if ~isfile(nome_file)
36         warning('File %s non trovato. Salto la matrice.', nome_file);
37         continue;
38     end
39
40     % Caricamento della matrice
41     data = load(nome_file);
42     if isfield(data, 'Problem')
43         A = data.Problem.A;
44     else
45         campi = fieldnames(data);
46         A = data.(campi{1});
47     end
48 end
```

```

48 matlab_pid = feature('getpid');
49 if ispc
50     cmd = sprintf('powershell -Command "(Get-Process -Id %d).WorkingSet64"',
51 matlab_pid);
52     [~, mem_str] = system(cmd);
53     mem_iniziale = str2double(strtrim(mem_str)) / 1024^2;
54 else
55     cmd = sprintf('ps -p %d -o rss=', matlab_pid);
56     [~, mem_str] = system(cmd);
57     mem_iniziale = str2double(strtrim(mem_str)) / 1024;
58 end
59
60 N = size(A, 1);
61
62 % --- SETUP MONITORAGGIO MEMORIA ---
63 % Ottieni il Process ID (PID) di questa sessione di MATLAB
64 matlab_pid = feature('getpid');
65
66 % Definisci i percorsi per i file di scambio
67 log_file = fullfile(pwd, 'mem_log.txt');
68 flag_file = fullfile(pwd, 'stop_monitor.txt');
69
70 % Assicurati che i file della sessione precedente siano eliminati
71 if isfile(log_file), delete(log_file); end
72 if isfile(flag_file), delete(flag_file); end
73
74 % Lancia lo script in background in base al Sistema Operativo
75 if ispc
76     % Windows: Usa Start /B per lanciare PowerShell in background senza aprire
77     % nuove finestre
78     cmd = sprintf('start /B powershell -ExecutionPolicy Bypass -WindowStyle Hidden
79     -File monitor_memory.ps1 -PIDToMonitor %d -LogFile "%s" -FlagFile "%s"',
80 matlab_pid, log_file, flag_file);
81     system(cmd);
82 else
83     % Linux: Esegue lo script bash in background usando la e commerciale (&)
84     cmd = sprintf('./monitor_memory.sh %d "%s" "%s" &', matlab_pid, log_file,
85 flag_file);
86     system(cmd);
87 end
88
89 % Aspetta che il file di log venga creato e che contenga dei dati (massimo 15 sec)
90 timeout = 15;
91 t_wait = tic;
92 log_pronto = false;
93
94 while toc(t_wait) < timeout
95     if isfile(log_file)
96         % Se il file esiste, controlla che non sia vuoto (bytes > 0)
97         info_file = dir(log_file);
98         if info_file.bytes > 0
99             log_pronto = true;
100             break; % Usciamo dal ciclo: PowerShell ha iniziato a scrivere!
101         end
102     end
103     pause(0.1); % Controlla ogni decimo di secondo
104 end
105
106 if ~log_pronto
107     warning('Timeout: lo script esterno e' troppo lento o non si e' avviato.');
```

```

114 % Crea il file "flag" per dire allo script in background di fermarsi
115 fid = fopen(flag_file, 'w');
116 fclose(fid);
117
118 % Breve pausa per dare tempo allo script di accorgersi del flag e chiudersi
119 pause(0.5);
120
121 % --- CALCOLO INCREMENTO RAM ---
122 if isfile(log_file)
123     mem_data = readmatrix(log_file); % Legge l'array dei campionamenti in KB
124     if ~isempty(mem_data)
125         mem_peak = max(mem_data) / 1024; % Picco di memoria raggiunto
126         % Incremento in Megabyte (MB)
127         memoria = mem_peak - mem_iniziale;
128         memorie(i) = round(memoria,4);
129     else
130         memorie(i) = 0;
131         warning('Il file di log della memoria e vuoto per %s.', matrici_names{i});
132     end
133 else
134     memorie(i) = 0;
135     warning('Impossibile trovare il file di log della memoria per %s.',
136     matrici_names{i});
137 end
138
139 N_vals(i) = N;
140 fprintf("N: %-8d | ", N_vals(i));
141
142 % Stampa dei risultati intermedi
143 fprintf('Tempo: %8.4f s | Errore: %8.4e | Incremento Memoria: %8.4f MB\n', ...
144     tempi(i), errori(i), memorie(i));
145
146 % Pulizia per liberare la RAM prima del ciclo successivo
147 clear A data;
148
149 % Rimuove i file temporanei
150 if isfile(log_file), delete(log_file); end
151 if isfile(flag_file), delete(flag_file); end
152 end
153
154 fprintf('-----\n');
155 fprintf('Analisi completata. Generazione dei grafici...\n');
156
157 % Ordinamento dei dati in base alla dimensione N per avere un grafico leggibile (linea
158 % continua da N piccolo a N grande)
159 [N_vals_sorted, idx] = sort(N_vals);
160 tempi_sorted = tempi(idx);
161 errori_sorted = errori(idx);
162 memorie_sorted = memorie(idx);
163
164 % Esclusione eventuali file saltati perche' non trovati (N = 0)
165 valid_idx = N_vals_sorted > 0;
166 N_vals_sorted = N_vals_sorted(valid_idx);
167 tempi_sorted = tempi_sorted(valid_idx);
168 errori_sorted = errori_sorted(valid_idx);
169 memorie_sorted = memorie_sorted(valid_idx);
170
171 % --- CREAZIONE DEI GRAFICI ---
172 figure('Name', 'Prestazioni MATLAB: Cholesky su Matrici Sparse', 'Position', [100,
173     100, 1200, 400]);
174
175 % Grafico 1: Tempo
176 subplot(1, 3, 1);
177 semilogy(N_vals_sorted, tempi_sorted, '-o', 'LineWidth', 2, 'MarkerFaceColor', 'b');
178 title('Tempo di Risoluzione');
179 xlabel('Dimensione matrice (N)');
180 ylabel('Tempo (secondi)');
181 grid on;
182
183 % Grafico 2: Errore Relativo
184 subplot(1, 3, 2);

```

```

182 semilogy(N_vals_sorted, errori_sorted, '-s', 'LineWidth', 2, 'MarkerFaceColor', 'r', '
    Color', 'r');
183 title('Errore Relativo');
184 xlabel('Dimensione matrice (N)');
185 ylabel('Errore relativo');
186 grid on;
187
188 % Grafico 3: Memoria
189 subplot(1, 3, 3);
190 semilogy(N_vals_sorted, memorie_sorted, '-~', 'LineWidth', 2, 'MarkerFaceColor', 'g',
    'Color', 'g');
191 title('Incremento di Memoria RAM');
192 xlabel('Dimensione matrice (N)');
193 ylabel('Memoria (MB)');
194 grid on;
195
196 % --- SALVATAGGIO DATI IN CSV ---
197 nomi_sorted = matrici_names(idx); % Riordina usando gli indici del sort
198 nomi_sorted = strrep(nomi_sorted(valid_idx)', '.mat', ''); % Filtra i validi e
    traspone in colonna
199
200 % Creazione della tabella riassuntiva
201 risultati_tabella = table(nomi_sorted, N_vals_sorted, tempi_sorted, errori_sorted,
    memorie_sorted, ...
202     'VariableNames', {'Matrice', 'Dimensione (N)', 'Tempo (s)', 'Errore relativo', '
    Memoria (MB)'});
203
204 % Scrittura su file
205 if ispc
206     nome_file_csv = 'risultati_matlab_windows.csv';
207 else
208     nome_file_csv = 'risultati_matlab_linux.csv';
209 end
210
211 % ottieni la cartella genitore della current folder
212 parentFolder = fileparts(pwd);
213
214 folderName = 'results';
215
216 if ~isfolder(fullfile(parentFolder, folderName))
217     mkdir(parentFolder, folderName);
218 end
219
220 % costruisci il percorso alla sottocartella results nella cartella genitore
221 folder = fullfile(parentFolder, folderName);
222
223 writetable(risultati_tabella, fullfile(folder, nome_file_csv));
224
225 fprintf('\nI risultati sono stati salvati con successo nel file: %s\n', nome_file_csv)
    ;

```

```

1 function [error] = solve(A, N)
2     % Questa funzione isola la risoluzione del sistema lineare per calcolare
3     % accuratamente l'aumento di memoria dedicato solo a questa operazione.
4
5     % Creazione del vettore soluzione esatta e termine noto
6     xe = ones(N, 1);
7     b = A * xe;
8
9     % Risoluzione e tempo
10    x = A \ b;
11
12    error = norm(x - xe, 2) / norm(xe, 2);
13 end

```

Campionamento e Monitoraggio Asincrono della RAM

La misurazione dell'incremento reale di memoria è affidata a una strategia ibrida che combina MATLAB con strumenti nativi del sistema operativo, superando i limiti della Garbage Collection interna:

- **RAM Iniziale:** All'inizio di ogni ciclo, lo script interroga il sistema operativo tramite comandi `system` per registrare l'occupazione di memoria base del processo MATLAB tramite il suo PID.
- **Esecuzione in background:** Subito dopo, viene lanciato in modalità asincrona uno script esterno.
- **Sincronizzazione e Picco:** Al termine del calcolo lineare, MATLAB crea il file di flag `stop_monitor.txt` per arrestare lo script esterno. Successivamente, legge il log, estrae il valore massimo e calcola l'incremento netto rispetto al valore iniziale.

Calcolo delle metriche

La risoluzione del sistema è isolata nella funzione locale `solve(A, N)` per circoscrivere l'ambiente di calcolo:

- **Risoluzione:** All'interno della funzione viene definito il vettore soluzione esatta x_e , calcolato il termine noto coerente $b = A \cdot x_e$ ed eseguito l'operatore backslash (`A \b`).
- **Tempo e Accuratezza:** Il tempo netto di calcolo viene misurato strettamente attorno alla routine `solve` tramite i comandi `tic/toc`.

Post-Elaborazione ed Esportazione dei dati

Concluso il ciclo principale, lo script esegue le seguenti operazioni di finalizzazione:

- **Ordinamento:** I risultati vengono ordinati in base alla dimensione spaziale delle matrici (N), per permettere alle funzioni `semilogy` di tracciare curve continue e coerenti al crescere della complessità del problema.
- **Esportazione CSV:** I dati ordinati vengono strutturati in una tabella MATLAB ed esportati automaticamente in formato CSV all'interno della cartella `results`.

6.1.2 Profiler

```

1  # monitor_memory.ps1
2  param (
3      [int]$PIDToMonitor,
4      [string]$LogFile,
5      [string]$FlagFile
6  )
7
8  # Pulisce il log precedente
9  Remove-Item -Path $LogFile -ErrorAction SilentlyContinue
10
11 # Continua a registrare finche' non viene creato il file "FLAGFILE" da MATLAB
12 while (-not (Test-Path -Path $FlagFile)) {
13     $process = Get-Process -Id $PIDToMonitor -ErrorAction SilentlyContinue
14     if ($process) {
15         # Salva la memoria in KB
16         $memKB = [math]::Round($process.WorkingSet64 / 1024)
17         $memKB | Out-File -FilePath $LogFile -Append
18     }
19     Start-Sleep -Milliseconds 5 # Campiona ogni 5ms
20 }

```

Listing 6.1: Script di profiling per ambiente Windows (`monitor_memory.ps1`)

```

1  #!/bin/bash
2  # monitor_memory.sh
3  PID=$1
4  LOGFILE=$2
5  FLAGFILE=$3
6
7  # Pulisce il log precedente
8  > "$LOGFILE"

```



```

9
10 # Continua a registrare finché non viene creato il file "FLAGFILE" da MATLAB
11 while [ ! -f "$FLAGFILE" ]; do
12     # Estrae la memoria RSS in KB per il PID e la salva nel log
13     ps -p $PID -o rss= >> "$LOGFILE"
14     sleep 0.005 # Campiona ogni 5ms
15 done
16
17 # Lancia chmod +x monitor_memory.sh da terminale prima di avviare MATLAB

```

Listing 6.2: Script di profiling per ambiente Linux (monitor_memory.sh)

Evidenziato da test effettuati durante il benchmark, le funzioni diagnostiche interne di MATLAB (come la funzione `memory` o il profiler standard) risultano inadeguate per misurare il reale picco di occupazione della memoria durante l'invocazione di solutori diretti precompilati in C, come `CHOLMOD`.

Windows

In ambiente Microsoft Windows, il monitoraggio è demandato a uno script scritto in linguaggio **PowerShell**. Il processo riceve in input tre parametri: il Process ID (PID) della sessione di MATLAB, il percorso del file di log su cui scrivere e il percorso del file di flag usato come segnale di terminazione. Lo script invoca la cmdlet `Get-Process` puntando al PID di MATLAB per estrarre la proprietà di sistema **WorkingSet64**. Il valore originario, espresso in byte, viene diviso per 1024 per ottenere il dato in Kilobyte (KB) e successivamente appeso al file di log testuale.

Linux

La controparte per i sistemi Unix-like è stata realizzata tramite uno script **Bash**. L'architettura logica è speculare a quella di Windows, ma l'estrazione del dato si affida agli strumenti nativi del kernel Linux. Per isolare il consumo di RAM del processo MATLAB, lo script utilizza il comando POSIX `ps -p $PID -o rss=`.

Questo comando estrae in modo diretto e formattato la metrica RSS (Resident Set Size). Il valore RSS nativo restituito da `ps` è già formattato in Kilobyte (KB), permettendo di scriverlo direttamente all'interno del file di log senza ulteriori conversioni matematiche.

6.2 Python

```

1 def solve_matrix(A, b, xe):
2     factor = cholesky(A)
3     x = factor(b)
4     relative_error = np.linalg.norm(x - xe) / np.linalg.norm(xe)
5
6     return relative_error
7
8 def solve_and_measure(mtx_file):
9     results = {'matrix': os.path.splitext(os.path.basename(mtx_file))[0]}
10
11     # Lettura da file MTX
12     BASE_DIR = os.path.dirname(os.path.abspath(__file__))
13     mtx_path = os.path.join(BASE_DIR, mtx_file)
14     if os.path.exists(mtx_path):
15         A = csc_matrix(mmread(mtx_path))
16         n = A.shape[0]
17         xe = np.ones(n)
18         b = A @ xe
19
20         results['shape'] = A.shape[0]
21     else:
22         print("No matrix file found. Please provide a .mtx file or modify the code to
23         generate a matrix.")
24         return results
25
26     # Decomposizione di Choleski + risoluzione sistema A*x = b
27     try:

```

```

27         t0 = time.perf_counter()
28         relative_error = solve_matrix(A, b, xe)
29         time_ms = time.perf_counter() - t0
30
31         mem_history = memory_usage((solve_matrix, (A, b, xe)), interval=0.005, timeout
= None, include_children=True )
32
33         if mem_history:
34             peak_memory = max(mem_history)
35             mem_used = peak_memory - mem_history[0]
36
37             results['peak_memory'] = f"{mem_used:.4f}"
38             results['time'] = f"{time_ms:.4f}"
39             results['relative_error'] = f"{relative_error:.4e}"
40
41             return results
42
43     except Exception as e:
44         print(f"ERRORE: {e}")
45         results['error'] = str(e)
46
47 if __name__ == '__main__':
48     matrix_files = [
49         "./matrices/ex15.mtx",
50         "./matrices/cfd1.mtx",
51         "./matrices/shallow_water1.mtx",
52         "./matrices/cfd2.mtx",
53         "./matrices/parabolic_fem.mtx",
54         "./matrices/apache2.mtx",
55         "./matrices/G3_circuit.mtx",
56         "./matrices/StocF-1465.mtx",
57         "./matrices/Flan_1565.mtx"
58     ]
59
60     matrix_ordered = []
61     shapes = {}
62
63     for f in matrix_files:
64         BASE_DIR = os.path.dirname(os.path.abspath(__file__))
65         mtx_path = os.path.join(BASE_DIR, f)
66         if os.path.exists(mtx_path):
67             A = csc_matrix(mread(mtx_path))
68             shapes[f] = A.shape
69         else:
70             print(f"File {f} non trovato.")
71             shapes[f] = (0, 0)
72
73     matrix_ordered = sorted(matrix_files, key=lambda x: shapes[x][0])
74
75     ROOT_DIR = os.path.dirname(BASE_DIR)
76     RESULTS_DIR = os.path.join(ROOT_DIR, 'results')
77     if not os.path.exists(RESULTS_DIR):
78         os.makedirs(RESULTS_DIR)
79
80     os_name = platform.system()
81     if os_name == 'Windows':
82         csv_file = os.path.join(RESULTS_DIR, 'risultati_python_windows.csv')
83     else:
84         csv_file = os.path.join(RESULTS_DIR, 'risultati_python_linux.csv')
85
86     with open(csv_file, 'w', newline='') as csvfile:
87         writer = csv.writer(csvfile)
88         writer.writerow(['Matrice', 'Dimensione (N)', 'Tempo (s)', 'Errore relativo',
89 'Memoria (MB)'])
90
91     for f in matrix_ordered:
92         result = solve_and_measure(f)

```

Caricamento delle matrici e preparazione del problema

La funzione `solve_and_measure(mtx_file)` gestisce l'intero processo di benchmark per ciascuna matrice del dataset.

- **Lettura da file MTX:** La matrice sparsa viene caricata dal file Matrix Market tramite `mmread` e convertita nel formato compresso per colonne (`csc_matrix`).
- **Costruzione del sistema lineare:** Una volta determinata la dimensione del problema N , viene definito il vettore soluzione esatta x_e . Il termine noto viene quindi generato come $b = A \cdot x_e$, ottenendo un sistema con soluzione nota che consente di valutare accuratamente l'errore numerico.

Decomposizione di Cholesky e calcolo delle metriche

La risoluzione del sistema è isolata nella funzione `solve_matrix()`.

- **Fattorizzazione:** La funzione `cholesky()` calcola la decomposizione di Cholesky della matrice sparsa simmetrica definita positiva. L'oggetto restituito rappresenta la fattorizzazione e viene successivamente utilizzato per risolvere il sistema lineare.
- **Risoluzione:** Tramite la chiamata `factor(b)` viene ottenuta la soluzione numerica x del sistema $A \cdot x = b$.
- **Errore relativo:** L'accuratezza della soluzione viene valutata mediante la norma relativa

$$\frac{\|x - x_e\|}{\|x_e\|}$$

calcolata tramite le funzioni di algebra lineare di NumPy.

- **Tempo di esecuzione:** Il tempo necessario alla fattorizzazione e alla risoluzione viene misurato mediante `time.perf_counter()`, che fornisce un timer ad alta risoluzione indipendente dalla piattaforma.

Monitoraggio della memoria

La misurazione dell'utilizzo di memoria viene effettuata tramite la libreria `memory_profiler`. La funzione `memory_usage()` esegue la routine `solve_matrix()` monitorando il processo a intervalli di 5 ms (`interval=0.005`). Al termine dell'esecuzione viene estratto il massimo valore registrato durante il monitoraggio. Il consumo effettivo di memoria viene ottenuto sottraendo al valore di picco il primo campione acquisito, corrispondente alla memoria occupata prima dell'esecuzione dell'algoritmo.

Ordinamento ed esportazione dei risultati

Nella sezione principale dello script (`__main__`) vengono eseguite le operazioni finali di gestione dei dati.

Tutte le matrici del dataset vengono preliminarmente caricate per determinarne la dimensione. L'elenco viene quindi ordinato in base alla dimensione della matrice, garantendo una rappresentazione dei risultati al crescere della complessità del problema. Se la directory `results` non è presente, viene generata automaticamente.

Lo script identifica il sistema operativo tramite `platform.system()` e seleziona automaticamente il file di output appropriato (`risultati_python_windows.csv` oppure `risultati_python_linux.csv`). I risultati vengono infine salvati in formato CSV contenente nome della matrice, dimensione, tempo di esecuzione, errore relativo e memoria utilizzata.

Bibliography

- [1] Yanqing Chen, Tim Davis, William Hager, and Siva Rajamanickam. Algorithm 887: Cholmod, supernodal sparse cholesky factorization and update/downdate. ACM Trans. Math. Softw., 35, 01 2008.
- [2] mldivide - Solve systems of linear equations $Ax = B$ for x - MATLAB. URL <https://it.mathworks.com/help/matlab/ref/double.mldivide.html>.
- [3] Ronald F. Boisvert, Roldan Pozo, Karin Remington, Richard F. Barrett, and Jack J. Dongarra. Matrix Market: a web resource for test matrix collections, pages 125–137. Springer US, Boston, MA, 1997. ISBN 978-1-5041-2940-4. doi: 10.1007/978-1-5041-2940-4_9. URL https://doi.org/10.1007/978-1-5041-2940-4_9.